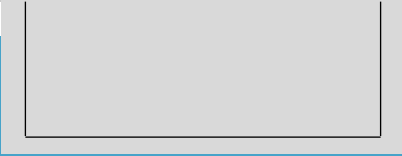




# *Manipuler l'ORM Eloquent*



# *INTÉRÊT DE L'OBJECT RELATIONAL MAPPER ORM ELOQUENT*

# *Qu'est-ce que Laravel Eloquent ?*

L'ORM (Object Relational Mapper) Eloquent de Laravel est une manière élégante, belle et efficace d'interagir et de gérer les bases de données.

L'équipe Laravel a créé Eloquent pour faciliter l'interaction et la communication avec les bases de données et fournir un moyen facile à utiliser pour résoudre tous les problèmes liés aux bases de données et au développement.

Laravel Eloquent offre la liberté d'écrire des codes personnalisables, bien formatés, efficaces, faciles à lire, bien documentés et durables en suivant les normes et les meilleures pratiques de l'industrie.

# *Aperçu de Laravel Eloquent (comment ça marche)*

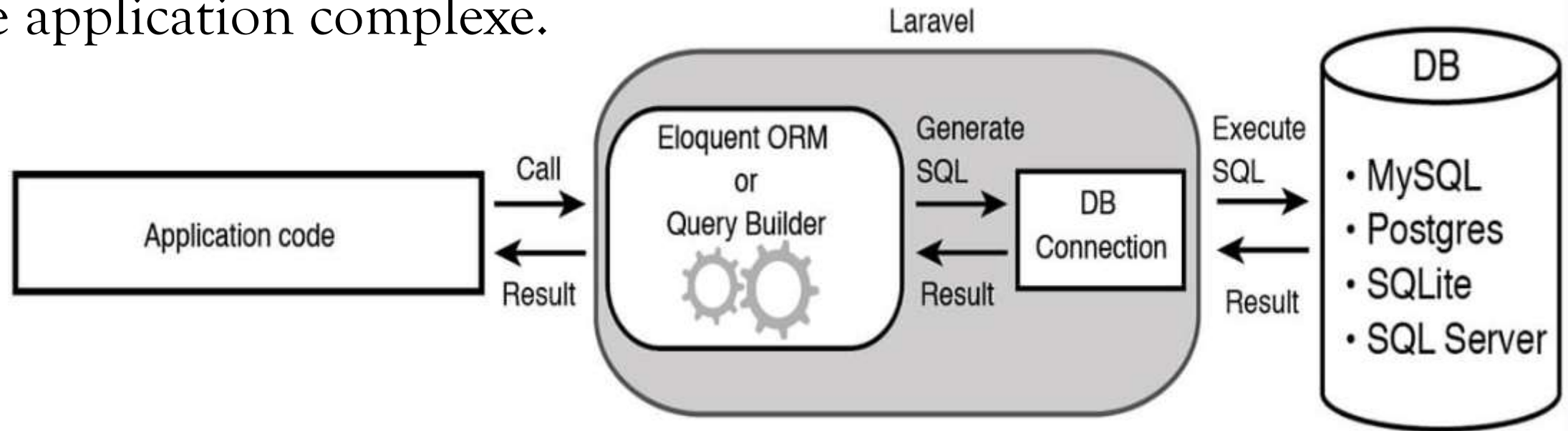
Laravel Eloquent utilise la mise en œuvre d'ActiveRecord, ce qui permet de travailler avec plusieurs bases de données en un clin d'œil.

Active Record est un modèle architectural qui structure chaque modèle Eloquent dans l'architecture MVC pour correspondre à la table dans la base de données.

En outre, vous pouvez facilement créer des données relationnelles dans votre base de données et travailler avec des modèles orientés objet de manière transparente en utilisant Laravel Eloquent.

# *Aperçu de Laravel Eloquent (comment ça marche)*

De plus, Laravel Eloquent vous donne la liberté de créer des schémas de base de données complexes et des interactions avec la base de données sans écrire et utiliser des requêtes SQL complexes ou longues, car les requêtes SQL sont fastidieuses et longues à apprendre et à coder dans une application complexe.



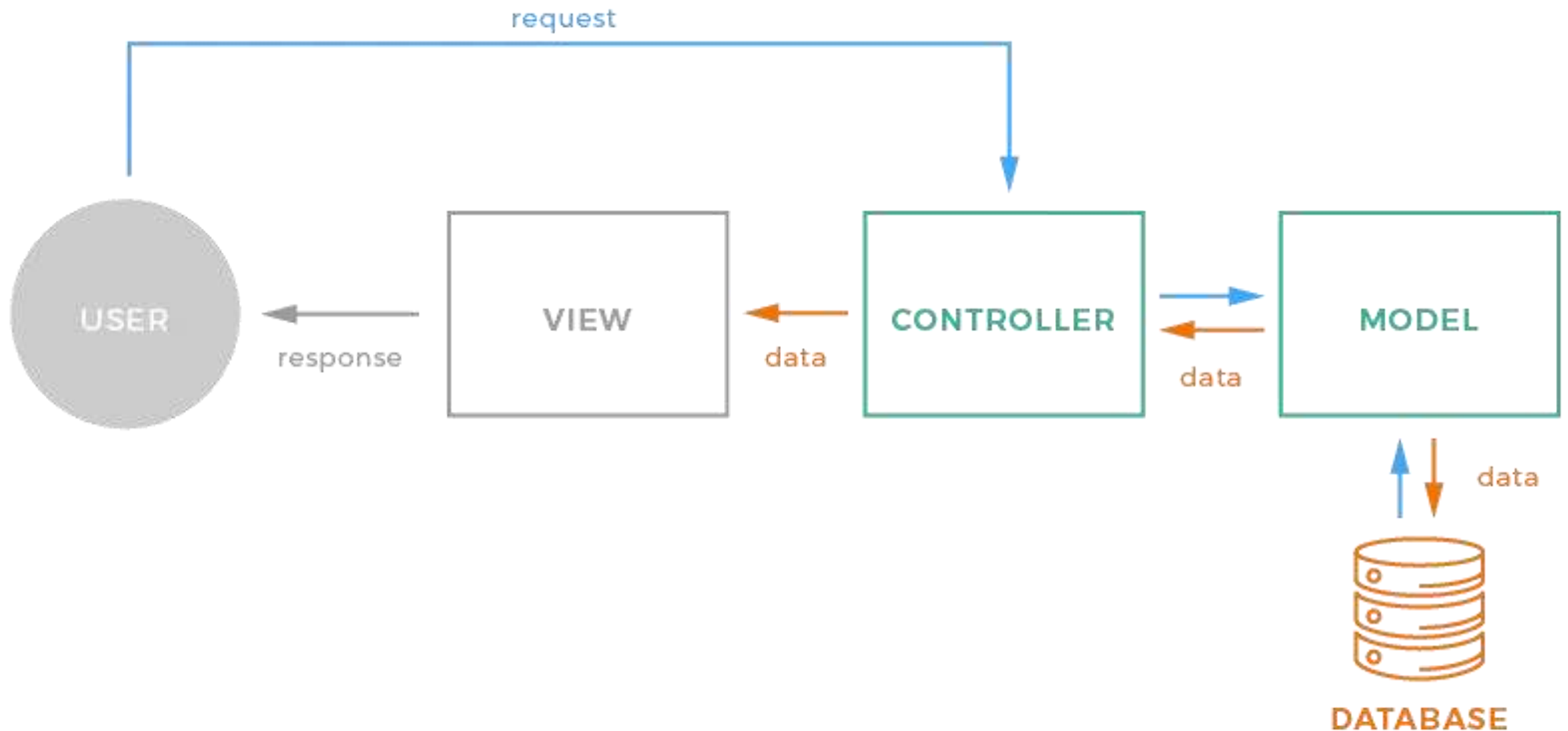
# *Qu'est-ce qu'un modèle Laravel ?*

Laravel Model est un objet PHP unique qui représente un schéma de base de données dans une base de données.

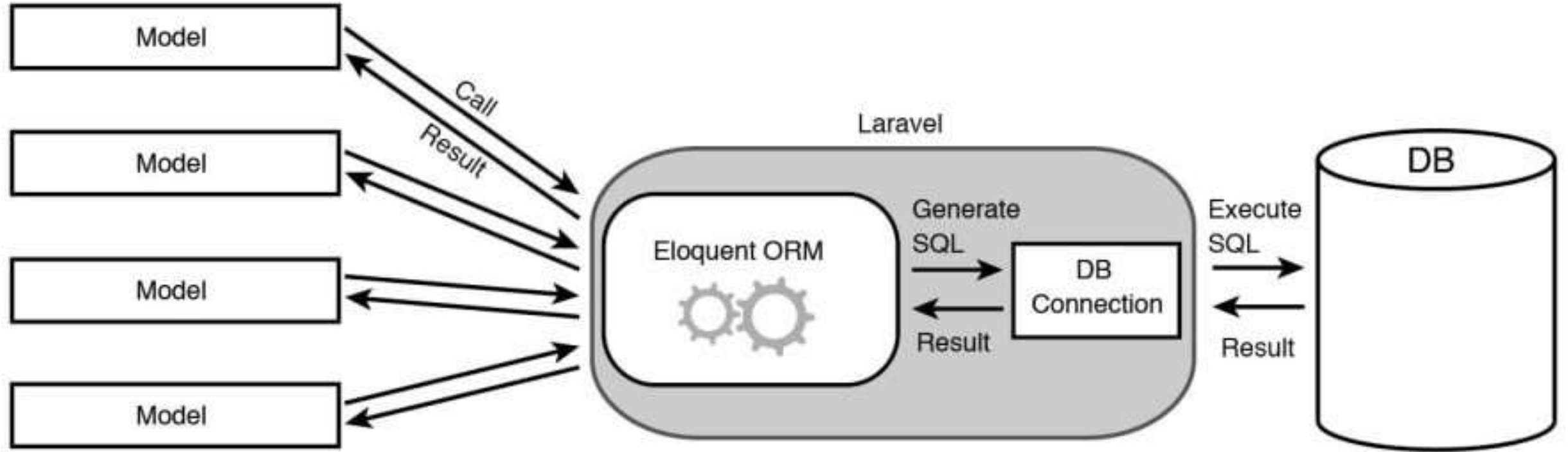
Le modèle vous aide à interagir et à communiquer avec un schéma de base de données spécifique.

En utilisant le modèle Eloquent, vous pouvez exécuter de nombreuses fonctionnalités liées à la base de données, telles que la création, la mise à jour, la lecture et la suppression d'enregistrements.

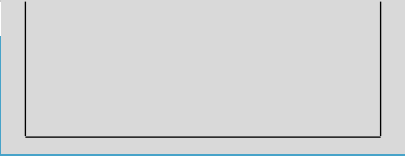
# *Qu'est-ce qu'un modèle Laravel ?*



# *Qu'est-ce qu'un modèle Laravel ?*



...



# *MANIPULATION DES MODÈLES*

# *Génération des classes de modèle*

Pour commencer, nous allons créer un modèle Eloquent. Les modèles se trouvent généralement dans le répertoire `app\Models` et étendent la classe `Illuminate\Database\Eloquent\Model`. Vous pouvez utiliser la commande Artisan `make:model` pour générer un nouveau modèle :

```
php artisan make:model Book
```

## *Conventions: Noms des tables*

Vous avez peut-être remarqué que nous n'avons pas indiqué à Eloquent quelle table de base de données correspond à notre modèle **Book**. Par convention, le nom pluriel, "snake case", de la classe sera utilisé comme nom de table, à moins qu'un autre nom ne soit explicitement spécifié.

Dans ce cas, Eloquent supposera que le modèle **Book** stocke les enregistrements dans la table **books**, tandis qu'un modèle **CommandDetails** stockerait les enregistrements dans une table **command\_details**.

## *Conventions: Clés primaires*



Eloquent suppose également que la table de base de données correspondante de chaque modèle possède une colonne de clé primaire nommée **id**. Si nécessaire, vous pouvez définir une propriété **\$primaryKey** protégée sur votre modèle pour spécifier une colonne différente qui sert de clé primaire à votre modèle.

```
protected $primaryKey = 'book_id';
```

## *Conventions: Timestamps*

Par défaut, Eloquent s'attend à ce que les colonnes **created\_at** et **updated\_at** existent sur la table de base de données correspondante de votre modèle. Eloquent définira automatiquement les valeurs de ces colonnes lorsque les modèles sont créés ou mis à jour. Si vous ne voulez pas que ces colonnes soient gérées automatiquement par Eloquent, vous devez définir une propriété **\$timestamps** sur votre modèle avec une valeur de **false**.

```
public $timestamps = false;
```

## *Conventions: Connexions aux bases de données*

Par défaut, tous les modèles Eloquent utiliseront la connexion de base de données par défaut qui est configurée pour votre application. Si vous souhaitez spécifier une connexion différente qui doit être utilisée lors de l'interaction avec un modèle particulier, vous devez définir une propriété **\$connection** sur le modèle.

```
protected $connection = 'sqlite';
```

## *Conventions: Valeurs par défaut des attributs*

Par défaut, une instance de modèle nouvellement instanciée ne contient aucune valeur d'attribut. Si vous souhaitez définir les valeurs par défaut de certains attributs de votre modèle, vous pouvez définir une propriété `$attributes` sur votre modèle.

```
protected $attributes = [  
    'options' => '[]',  
    'price' => 0,  
];
```

# *Récupération des modèles*

Une fois que vous avez créé un modèle et sa table de base de données associée, vous êtes prêt à commencer à récupérer des données dans votre base de données. La méthode **all** du modèle récupère tous les enregistrements de la table de base de données associée au modèle.

```
foreach (Book::all() as $book) {  
    echo $book>name;  
}
```

# *Construire des requêtes*

La méthode Eloquent **all** renverra tous les résultats dans la table du modèle. Cependant, puisque chaque modèle Eloquent sert de générateur de requêtes, vous pouvez ajouter des contraintes supplémentaires aux requêtes et ensuite invoquer la méthode **get** pour récupérer les résultats.

```
$books = Book::where('price', 0)
    ->orderBy('name')
    ->take(10)
    ->get();
```

## *Rafraîchir les modèles*

Si vous avez déjà une instance d'un modèle Eloquent qui a été récupéré dans la base de données, vous pouvez "rafraîchir" le modèle en utilisant les méthodes **fresh** et **refresh**. La méthode **fresh** récupère à nouveau le modèle dans la base de données. L'instance de modèle existante ne sera pas affectée.

```
$freshBook = $book>fresh();
```

La méthode **refresh** récupère le modèle existant en utilisant des données fraîches de la base de données.

```
$book>refresh();
```

# *Collections*

Comme nous l'avons vu, les méthodes Eloquent comme **all** et **get** récupèrent plusieurs enregistrements de la base de données. Cependant, ces méthodes ne renvoient pas un simple tableau PHP. Au lieu de cela, une instance de **Illuminate\Database\Eloquent\Collection** est retournée.

La classe Eloquent Collection étend la classe de base **Illuminate\Support\Collection** de Laravel, qui fournit une variété de méthodes utiles pour interagir avec les collections de données.

## *Découpage du résultats*

Votre application peut manquer de mémoire si vous tentez de charger des dizaines de milliers d'enregistrements Eloquent via les méthodes **all** ou **get**. Au lieu d'utiliser ces méthodes, la méthode **chunk** peut être utilisée pour traiter plus efficacement un grand nombre de modèles.



# *Découpage du résultats*

La méthode **chunk** récupère un sous-ensemble de modèles Eloquent, les transmettant à une fermeture pour traitement.

```
use App\Models\Book;

Book::chunk(200, function ($books) {
    foreach ($books as $book) {
        //
    }
});
```

## *Découpage à l'aide de Lazy Collections*

La méthode **lazy** fonctionne de la même manière que la méthode **chunk** dans le sens où, en coulisse, elle exécute la requête par morceaux. Cependant, au lieu de passer chaque morceau directement dans un callback, la méthode **lazy** renvoie une **LazyCollection** de modèles Eloquent, ce qui vous permet d'interagir avec les résultats en tant que flux unique.

```
foreach (Book::lazy() as $book) {  
    //  
}
```

## *Récupération des modèles uniques / agrégats*

Outre la récupération de tous les enregistrements correspondant à une requête donnée, vous pouvez également récupérer des enregistrements uniques à l'aide des méthodes **find**, **first** ou **firstWhere**. Au lieu de renvoyer une collection de modèles, ces méthodes renvoient une seule instance de modèle.

```
$book = Book::find(1);  
$book = Book::where('price', 0)->first();  
$book = Book::firstWhere('price', 0);
```

## *Récupération des modèles uniques / agrégats*

Parfois, vous pouvez souhaiter effectuer une autre action si aucun résultat n'est trouvé. Les méthodes **findOr** et **firstOr** renverront une seule instance de modèle ou, si aucun résultat n'est trouvé, exécuteront la fermeture donnée. La valeur renvoyée par la fermeture sera considérée comme le résultat de la méthode :

```
$book = Book::where('price', '>', 100)->firstOr(function () {  
    // ...  
});
```

# *Not Found Exceptions*

Parfois, vous pouvez souhaiter lever une exception si un modèle n'est pas trouvé. Les méthodes **findOrFail** et **firstOrFail** récupèrent le premier résultat de la requête ; toutefois, si aucun résultat n'est trouvé, une exception

**Illuminate\Database\Eloquent\ModelNotFoundException** est levée :

```
$book = Book::findOrFail(1);
```

```
$book = Book::where('price', '>', 100)->firstOrFail();
```

# *Récupérer ou créer des modèles*

La méthode **firstOrCreate** va tenter de localiser un enregistrement dans la base de données en utilisant les paires colonne/valeur données. Si le modèle ne peut être trouvé dans la base de données, un enregistrement sera inséré avec les attributs résultant de la fusion du premier argument de tableau avec le deuxième argument de tableau facultatif.

```
$book = Book::firstOrCreate([  
    'name' => '10 Roads to happiness'  
]);
```

## *Récupération des agrégats*

Lorsque vous interagissez avec les modèles Eloquent, vous pouvez également utiliser les méthodes **count**, **sum**, **max**, et autres méthodes d'agrégation fournies par le constructeur de requêtes de Laravel.

Comme vous pouvez vous y attendre, ces méthodes retournent une valeur scalaire au lieu d'une instance de modèle Eloquent :

```
$count = Book::where('rentable', 1)->count();
```

```
$max = Book::where('rentable', 1)->max('price');
```

# *Insertion de modèles*

Pour insérer un nouvel enregistrement dans la base de données, vous devez instancier une nouvelle instance de modèle et définir des attributs sur le modèle. Ensuite, appelez la méthode d'enregistrement sur l'instance de modèle

```
public function store(Request $request)
{
    // Validate the request...
    $book = new Book;
    $book->name = $request->name;
    $book->save();
}
```

## *Insertion de modèles*

Vous pouvez également utiliser la méthode **create** pour "enregistrer" un nouveau modèle en utilisant une seule instruction PHP. L'instance de modèle insérée vous sera retournée par la méthode **create**.

```
public function store(Request $request)
{
    // Validate the request...
    $book = Book::create([
        'name' => 'Book title',
    ]);
}
```

# *Mise à jour des modèles*

Pour mettre à jour un modèle, vous devez le récupérer et définir tous les attributs que vous souhaitez mettre à jour. Ensuite, vous devez appeler la méthode `save` du modèle.

```
public function update(Request $request)
{
    // Validate the request...
    $book = Book::find(1);
    $book->name = $request->name;
    $book->save();
}
```

# *Mise à jour des modèles*

Les mises à jour peuvent également être effectuées sur les modèles qui correspondent à une requête donnée.

```
Flight::where('rentable', 1)  
  ->where('author', 'Jack Spicer')  
  ->update(['price' => 0]);
```

# *Examen des changements d'attributs*

Eloquent fournit les méthodes **isDirty**, **isClean** et **wasChanged** pour examiner l'état interne de votre modèle et déterminer comment ses attributs ont changé depuis la récupération initiale du modèle.

La méthode **isDirty** détermine si l'un des attributs du modèle a été modifié depuis l'extraction du modèle.

La méthode **isClean** permet de déterminer si un attribut est resté inchangé depuis la récupération du modèle.

```
$user->isDirty();  
$user->isClean();  
$user->isDirty('title');  
$user->isClean('title');
```

# *Examen des changements d'attributs*

La méthode **wasChanged** détermine si des attributs ont été modifiés lors du dernier enregistrement du modèle dans le cycle de demande actuel.

```
$user->title = 'Painter';  
$user->save();
```

```
$user->wasChanged(); // true  
$user->wasChanged('title'); // true
```

# *Examen des changements d'attributs*

La méthode **getOriginal** renvoie un tableau contenant les attributs originaux du modèle, sans tenir compte des modifications apportées au modèle depuis sa récupération. Si nécessaire, vous pouvez passer un nom d'attribut spécifique pour obtenir la valeur originale d'un attribut particulier.

```
$user = User::find(1);  
$user->name; // John  
$user->name = "Jack";  
$user->getOriginal('name'); // John
```

# *Affectation en masse*

Avant d'utiliser la méthode **create**, vous devrez spécifier une propriété **fillable** ou **guarded** sur votre classe de modèle. Ces propriétés sont nécessaires car tous les modèles Eloquent sont protégés par défaut contre les vulnérabilités d'affectation en masse.

```
protected $fillable = ['name'];
```

```
// ou bien
```

```
protected $guarded = [];
```

## *Upserts (Update/Insert)*

Occasionnellement, vous pouvez avoir besoin de mettre à jour un modèle existant ou de créer un nouveau modèle si aucun modèle correspondant n'existe. Comme la méthode `firstOrCreate`, la méthode `updateOrCreate` assure la persistance du modèle, de sorte qu'il n'est pas nécessaire d'appeler manuellement la méthode `save`.

```
$book = Book::updateOrCreate(  
    ['name' => 'Oath of men', 'author' => 'Jack Spicer'],  
    ['price' => 99, 'discounted' => 1]  
);
```

# *Suppression de modèles*

Pour supprimer un modèle, vous pouvez appeler la méthode **delete** sur l'instance du modèle :

```
$book->delete();
```

Vous pouvez appeler la méthode **truncate** pour supprimer tous les enregistrements de la base de données associés au modèle.

```
Book::truncate();
```

## *Suppression d'un modèle existant par sa clé primaire*

Si vous connaissez la clé primaire du modèle, vous pouvez supprimer le modèle sans le récupérer explicitement en appelant la méthode **destroy**. En plus d'accepter une clé primaire unique, la méthode **destroy** accepte plusieurs clés primaires, un tableau de clés primaires ou une collection de clés primaires.

```
Book::destroy(1);  
Book::destroy(1, 2, 3);  
Book::destroy([1, 2, 3]);  
Book::destroy(collect([1, 2, 3]));
```

# *Suppression de modèles à l'aide de requêtes*

Vous pouvez construire une requête Eloquent pour supprimer tous les modèles correspondant aux critères de votre requête.

```
$deleted = Book::where('price', 0)->delete();
```

## *Suppression douce (Soft delete)*

En plus de la suppression effective des enregistrements de votre base de données, Eloquent peut également "soft delete" les modèles. Lorsque les modèles sont supprimés, ils ne sont pas réellement supprimés de votre base de données. Au lieu de cela, un attribut **deleted\_at** est défini sur le modèle indiquant la date et l'heure à laquelle le modèle a été "supprimé". Pour activer les suppressions progressives pour un modèle, ajoutez le trait `Illuminate\Database\Eloquent\SoftDeletes` au modèle.

```
use SoftDeletes;
```

## *Suppression douce (Soft delete)*

Lorsque vous appelez la méthode `delete` sur le modèle, la colonne `deleted_at` sera définie sur la date et l'heure actuelles. Lors de l'interrogation d'un modèle qui utilise des **Soft delete**, les modèles soft deletes seront automatiquement exclus de tous les résultats de l'interrogation.

Pour déterminer si une instance de modèle donnée a été supprimée par soft delete, vous pouvez utiliser la méthode "**trash**" :

```
if ($book->trashed()) {  
}
```

## *Restauration des modèles supprimés*

Il peut arriver que vous souhaitiez "annuler" un modèle supprimé. Pour restaurer un modèle supprimé, vous pouvez appeler la méthode **restore** sur une instance de modèle. La méthode **restore** donne la valeur **null** à la colonne **deleted\_at** du modèle :

```
$book->restore();
```

## *Restauration des modèles supprimés*

Vous pouvez également utiliser la méthode **restore** dans une requête pour restaurer plusieurs modèles. Encore une fois, comme pour les autres opérations "en masse", cette méthode n'enverra pas d'événements de modèle pour les modèles qui sont restaurés :

```
Book::withTrashed()  
    ->where('price', 100)  
    ->restore();
```

## *Suppression permanente des modèles*

Parfois, vous pouvez avoir besoin de supprimer réellement un modèle de votre base de données. Vous pouvez utiliser la méthode **forceDelete** pour supprimer définitivement un modèle **soft delete** de la table de la base de données.

```
$book->forceDelete();
```

## *Récupérer les modèles soft delete*

les modèles supprimés seront automatiquement exclus des résultats de la requête. Cependant, vous pouvez forcer l'inclusion des modèles supprimés en douceur dans les résultats d'une requête en appelant la méthode `withTrashed` sur la requête.

```
$books = Book::withTrashed()  
    ->where('price', 10)  
    ->get();
```

# *Récupérer les modèles soft delete*

La méthode `onlyTrashed` permet de récupérer uniquement les modèles supprimés en douceur.

```
$books = Book::onlyTrashed()  
    ->where('price', 10)  
    ->get();
```

## *Élagage des Modèles (Pruning)*

Il peut arriver que vous souhaitiez supprimer périodiquement les modèles qui ne sont plus nécessaires. Pour ce faire, vous pouvez ajouter le trait `Illuminate\Database\Eloquent\Prunable` ou `Illuminate\Database\Eloquent\MassPrunable` aux modèles que vous souhaitez élaguer périodiquement. Après avoir ajouté l'un de ces traits au modèle, implémentez une méthode **prunable** qui renvoie un constructeur de requête Eloquent qui résout les modèles qui ne sont plus nécessaires.

# *Élagage des Modèles (Pruning)*

```
use Illuminate\Database\Eloquent\Prunable;

class Book extends Model
{
    use Prunable;

    public function prunable()
    {
        return static::where('created_at', '<=', now()->subMonth());
    }
}
```

# *Réplication des modèles*

Vous pouvez créer une copie non sauvegardée d'une instance de modèle existante en utilisant la méthode **replicate**. Cette méthode est particulièrement utile lorsque vous avez des instances de modèle qui partagent un grand nombre d'attributs identiques.

```
$shipping = Address::create([  
    'type' => 'shipping',  
    'line_1' => '123 Example Street',  
    'city' => 'Victorville',  
    'state' => 'CA',  
    'postcode' => '90001',  
]);
```

```
$billing = $shipping->replicate()->fill([  
    'type' => 'billing'  
]);  
  
$billing->save();
```

# *Comparaison des modèles*

Il peut arriver que vous ayez besoin de déterminer si deux modèles sont "identiques" ou non. Les méthodes `is` et `isNot` peuvent être utilisées pour vérifier rapidement si deux modèles ont la même clé primaire, la même table et la même connexion à la base de données.

```
if ($post->is($anotherPost)) {  
    //  
}  
  
if ($post->isNot($anotherPost)) {  
    //  
}
```

# *Événements*

Les modèles Eloquent distribuent plusieurs événements, ce qui vous permet d'accéder aux moments suivants du cycle de vie d'un modèle :  
**retrieved, creating, created, updating, updated, saving, saved, deleting, deleted, trashed, forceDeleting, forceDeleted, restoring, restored, and replicating.**

## *Événements (Utilisation des fermetures)*

Au lieu d'utiliser des classes d'événements personnalisées, vous pouvez enregistrer des fermetures qui s'exécutent lorsque divers événements de modèle sont envoyés. En général, vous devez enregistrer ces fermetures dans la méthode "booted" de votre modèle.

```
protected static function booted()  
{  
    static::created(function ($user) {  
        //  
    });  
}
```

# *Les Observateurs*

Si vous écoutez de nombreux événements sur un modèle donné, vous pouvez utiliser des observateurs pour regrouper tous vos écouteurs dans une seule classe. Les classes d'observateurs ont des noms de méthodes qui reflètent les événements Eloquent que vous souhaitez écouter.

La commande **make:observer** est le moyen le plus simple de créer une nouvelle classe d'observateurs.

```
php artisan make:observer UserObserver --model=User
```

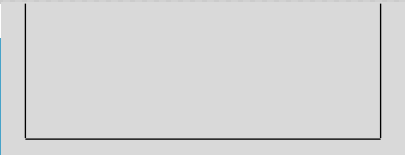
```
class UserObserver
{
    public function created(User $user)
    {
        //
    }
    public function updated(User $user)
    {
        //
    }
    public function deleted(User $user)
    {
        //
    }...
}
```

# *Les Observateurs*

Pour enregistrer un observateur, vous devez appeler la méthode **observe** sur le modèle que vous souhaitez observer. Vous pouvez enregistrer les observateurs dans la méthode **boot** du fournisseur de service **App\Providers\EventServiceProvider** de votre application :

```
use App\Models\User;
use App\Observers\UserObserver;

public function boot()
{
    User::observe(UserObserver::class);
}
```



# *RELATIONS ELOQUENT*

# *Les relations*

Les tables de la base de données sont souvent liées les unes aux autres. Eloquent facilite la gestion et le travail avec ces relations, et supporte une variété de relations courantes:

One To One

One To Many

Many To Many

Has One Through

Has Many Through

One To One (Polymorphic)

One To Many (Polymorphic)

Many To Many (Polymorphic)

## *Définir les relations*

Les relations Eloquent sont définies comme des méthodes sur vos classes de modèle Eloquent. Puisque les relations servent également de puissants constructeurs de requêtes, la définition des relations en tant que méthodes fournit de puissantes capacités de chaînage de méthodes et de requêtes.

## *Les relations (1:1)*

Une relation 1:1 est un type très basique de relation de base de données. Par exemple, un modèle **User** peut être associé à un modèle **Phone**. Pour définir cette relation, nous allons placer une méthode **phone** sur le modèle **User**. La méthode **phone** doit appeler la méthode **hasOne** et renvoyer son résultat.

```
class User extends Model
{
    public function phone()
    {
        return $this->hasOne(Phone::class);}
}
```

## *Les relations (1:1)*

Une fois la relation définie, nous pouvons récupérer l'enregistrement lié en utilisant les propriétés dynamiques d'Eloquent. Les propriétés dynamiques vous permettent d'accéder aux méthodes de relation comme si elles étaient des propriétés définies sur le modèle :

```
$phone = User::find(1)->phone;
```

## *Les relations (1:1)*

Eloquent détermine la clé étrangère de la relation sur la base du nom du modèle parent. Dans ce cas, le modèle **Phone** est automatiquement supposé avoir une clé étrangère **user\_id**. Si vous souhaitez passer outre cette convention, vous pouvez passer un deuxième argument à la méthode **hasOne**:

```
return $this->hasOne(Phone::class, 'foreign_key');
```

## *Définir l'inverse de la relation (1:1)*

Ainsi, nous pouvons accéder au modèle **Phone** à partir de notre modèle **User**. Ensuite, nous allons définir une relation sur le modèle **Phone** qui nous permettra d'accéder à l'utilisateur qui possède le téléphone. Nous pouvons définir l'inverse d'une relation **hasOne** en utilisant la méthode **belongsTo**.

```
class Phone extends Model
{
    public function user()
    {
        return $this->belongsTo(User::class);
    }
}
```

## *Les relations (1:n) - One To Many*

Une relation de type "un à plusieurs" est utilisée pour définir des relations où un modèle unique est le parent d'un ou de plusieurs modèles enfants.

```
class Post extends Model
{

    public function comments()
    {
        return $this->hasMany(Comment::class);
    }
}
```

## *Les relations (1:n) - One To Many*

Une fois que la méthode de relation a été définie, nous pouvons accéder à la collection de commentaires liés en accédant à la propriété **comments**.

```
use App\Models\Post;

$comments = Post::find(1)->comments;

foreach ($comments as $comment) {
    //
}
```

## *Les relations (1:n) - One To Many*

Étant donné que toutes les relations servent également de constructeurs de requêtes **QueryBuilder**, vous pouvez ajouter des contraintes supplémentaires à la requête de la relation en appelant la méthode des commentaires et en continuant à enchaîner les conditions sur la requête.

```
$comment = Post::find(1)->comments()  
            ->where('title', 'foo')  
            ->first();
```

## *Les relations (1:n) - l'inverse*

Maintenant que nous pouvons accéder à tous les commentaires d'un article, définissons une relation permettant à un commentaire d'accéder à son article (Post) parent. Pour définir l'inverse d'une relation **hasMany**, définissez une méthode de relation sur le modèle enfant qui appelle la méthode **belongsTo**

```
class Comment extends Model
{
    public function post()
    {
        return $this->belongsTo(Post::class);
    }
}
```

## *Les relations (1:n) - l'inverse*

Une fois la relation définie, nous pouvons récupérer le **post** parent d'un commentaire en accédant à la "propriété de relation dynamique" du commentaire :

```
use App\Models\Comment;  
  
$comment = Comment::find(1);  
  
return $comment->post->title;
```

## *Les relations (1:n) - l'inverse*

Les relations **belongsTo**, **hasOne** vous permettent de définir un modèle par défaut qui sera renvoyé si la relation donnée est nulle.

Dans l'exemple suivant, la relation **user** renverra un modèle

**App\Models\User** vide si aucun utilisateur n'est attaché au modèle **Post** :

```
public function user()  
{  
    return $this->belongsTo(User::class)->withDefault();  
}
```

## *Requête sur les relations Belongs To*

Lors de la recherche des enfants d'une relation "appartient à". Vous pouvez utiliser la méthode **whereBelongsTo**, qui déterminera automatiquement la relation et la clé étrangère appropriées pour le modèle donné.

```
$posts = Post::whereBelongsTo($user)->get();
```

// Ou

```
$posts = Post::whereBelongsTo($users)->get();
```

## *A un parmi plusieurs (has one of many)*

Parfois, un modèle peut avoir de nombreux modèles liés, mais vous voulez facilement récupérer le "dernier" ou le "plus ancien" modèle lié de la relation.

```
public function latestOrder()
{
    return $this->hasOne(Order::class)->latestOfMany();
}

public function oldestOrder()
{
    return $this->hasOne(Order::class)->oldestOfMany();
}
```

## *A un parmi plusieurs (has one of many)*

Par défaut, les méthodes `latestOfMany` et `oldestOfMany` récupèrent le modèle le plus récent ou le plus ancien.

Cependant, il arrive que vous souhaitiez extraire un seul modèle d'une relation plus importante en utilisant un critère de tri différent.

Par exemple, en utilisant la méthode `ofMany`, vous pouvez récupérer la commande la plus chère de l'utilisateur.

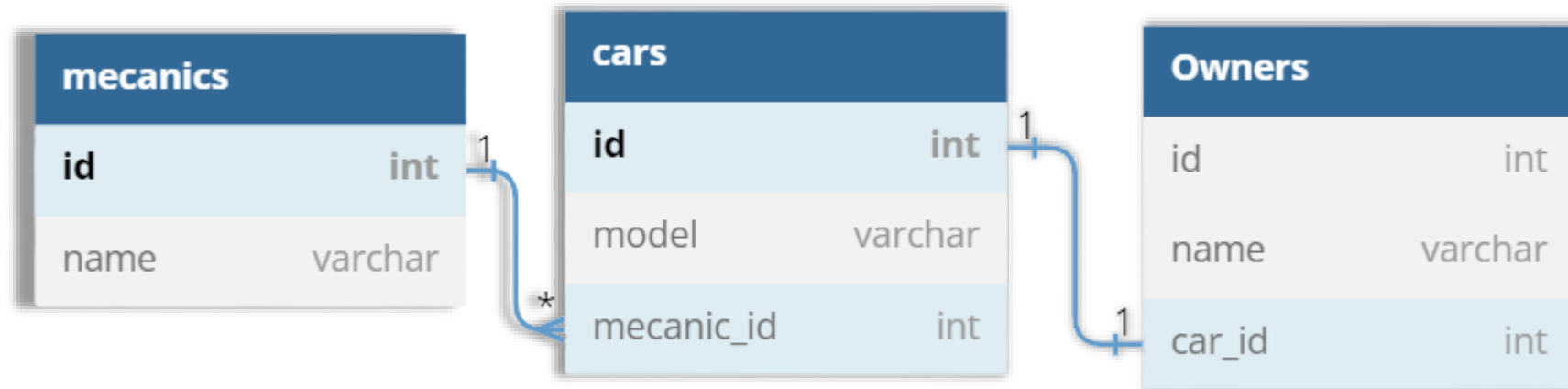
```
public function largestOrder()  
{  
    return $this->hasOne(Order::class)->ofMany('price', 'max');  
}
```

## *A un a travers (Has One Through)*

La relation "has-one-through" définit une relation univoque avec un autre modèle. Toutefois, cette relation indique que le modèle déclarant peut être mis en correspondance avec une instance d'un autre modèle en passant par un troisième modèle.

## *A un a travers (Has One Through)*

Par exemple, dans une application d'atelier de réparation de véhicules, chaque modèle Mécanicien ( **Mechanic** ) peut être associé à un modèle Voiture ( **Car** ), et chaque modèle Voiture peut être associé à un modèle Propriétaire ( **Owner** ). Bien que le mécanicien et le propriétaire n'aient pas de relation directe dans la base de données, le mécanicien peut accéder au propriétaire par le biais du modèle Voiture.



## *A un a travers (Has One Through)*

Maintenant que nous avons examiné la structure de la table pour la relation, définissons la relation sur le modèle **Mechanic** :

```
class Mechanic extends Model
{

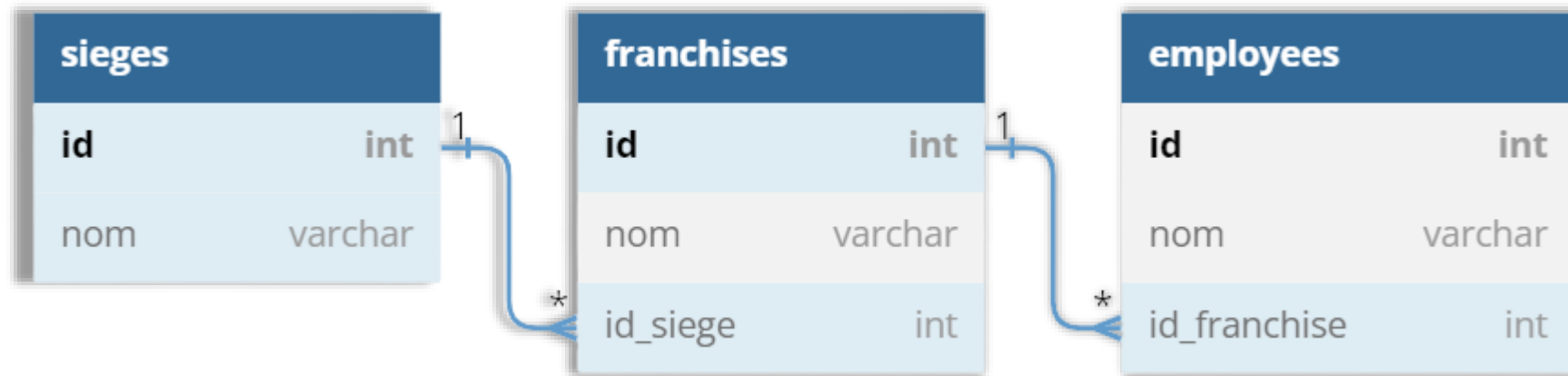
    public function carOwner()
    {
        return $this->hasOneThrough(Owner::class, Car::class);
    }

}
```

# *A plusieurs a travers (Has Many Through)*

La relation "has-many-through" offre un moyen pratique d'accéder à des relations distantes via une relation intermédiaire.

Par exemple, supposons que nous construisions une plateforme de société globale, un modèle **Siege** peut accéder à de nombreux modèles **Employee** via un modèle **Franchise** intermédiaire.



## *A plusieurs a travers (Has Many Through)*

Maintenant que nous avons examiné la structure de la table pour la relation, définissons la relation sur le modèle **Siege**:

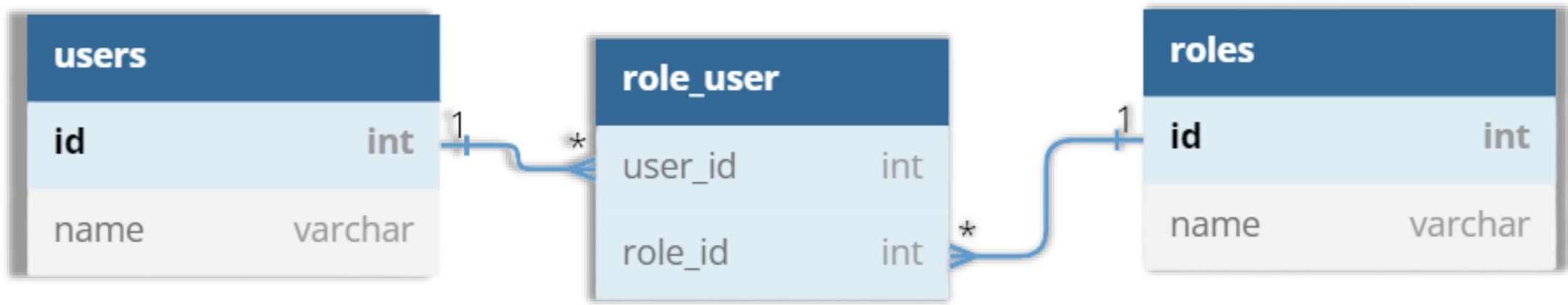
```
class Siege extends Model
{

    public function deployments()
    {
        return $this->hasManyThrough(Employee::class, Franchise::class);
    }

}
```

## *Relations plusieurs à plusieurs (n:n)*

Les relations **many-to-many** (plusieurs à plusieurs) sont légèrement plus compliquées que les relations **hasOne** et **hasMany**. Un exemple de relation **many-to-many** est un **utilisateur** qui a plusieurs **rôles** et ces **rôles** sont également partagés par d'autres **utilisateurs** de l'application.



## *Relations plusieurs à plusieurs (n:n)*

Les relations plusieurs à plusieurs sont définies en écrivant une méthode qui renvoie le résultat de la méthode **belongsToMany**.

```
class User extends Model
{

    public function roles()
    {
        return $this->belongsToMany(Role::class);
    }

}
```

## *Relations plusieurs à plusieurs (n:n)*

Une fois la relation définie, vous pouvez accéder aux rôles de l'utilisateur en utilisant la propriété de relation dynamique **roles** :

```
use App\Models\User;

$user = User::find(1);

foreach ($user->roles as $role) {
    //
}
```

## *Relations plusieurs à plusieurs (n:n)*

Pour déterminer le nom de la table intermédiaire de la relation, Eloquent joint les deux noms de modèles liés par **ordre alphabétique**. Cependant, vous êtes libre de passer outre cette convention. Vous pouvez le faire en passant un deuxième argument à la méthode `belongsToMany`

```
return $this->belongsToMany(Role::class, 'role_user');
```

## *Relations plusieurs à plusieurs (n:n)*

En plus de personnaliser le nom de la table intermédiaire, vous pouvez également personnaliser les noms des colonnes des clés de la table en passant des arguments supplémentaires à la méthode **belongsToMany**. Le troisième argument est le nom de la clé étrangère du modèle sur lequel vous définissez la relation, tandis que le quatrième argument est le nom de la clé étrangère du modèle que vous rejoignez.

```
return $this->belongsToMany(Role::class, 'role_user', 'user_id', 'role_id');
```

# *Relations plusieurs à plusieurs (n:n)*

## Récupération des colonnes de la table intermédiaire:

Eloquent fournit des moyens très utiles d'interagir avec la table intermédiaire. Chaque modèle que nous récupérons se voit automatiquement attribuer un attribut **pivot**. Cet attribut contient un modèle représentant la table intermédiaire.

```
$user = User::find(1);  
  
foreach ($user->roles as $role) {  
    echo $role->pivot->created_at;  
}
```

## *Relations plusieurs à plusieurs (n:n)*

Par défaut, seules les clés du modèle seront présentes sur le modèle **pivot**. Si votre tableau intermédiaire contient des attributs supplémentaires, vous devez les spécifier lors de la définition de la relation :

```
return $this->belongsToMany(Role::class)->withPivot('active', 'created_by');
```

Pour ajouter les timestamps:

```
return $this->belongsToMany(Role::class)->withTimestamps();
```

## *Relations plusieurs à plusieurs (n:n)*

Comme indiqué précédemment, les attributs de la table intermédiaire sont accessibles sur les modèles via l'attribut pivot. Cependant, vous êtes libre de personnaliser le nom de cet attribut pour mieux refléter son objectif dans votre application.

Exemple (user/podcast):

```
return $this->belongsToMany(Podcast::class)  
    ->as('subscription')  
    ->withTimestamps();
```

# *Relations polymorphes*

Une relation polymorphe permet au modèle enfant d'appartenir à plusieurs types de modèles à l'aide d'une seule association. Par exemple, imaginez que vous créez une application qui permet aux utilisateurs de partager des articles de blog et des vidéos. Dans une telle application, un modèle Commentaire peut appartenir à la fois aux modèles Article et Vidéo.

# *Relations polymorphes 1:1*

Une relation polymorphe un à un est similaire à une relation un à un typique ; toutefois, le modèle enfant peut appartenir à plusieurs types de modèles en utilisant une seule association.



## *Relations polymorphes 1:1*

La colonne **imageable\_id** contient la valeur d'identification du poste ou de l'utilisateur, tandis que la colonne **imageable\_type** contient le nom de la classe du modèle parent.

La colonne **imageable\_type** est utilisée par Eloquent pour déterminer le "type" de modèle parent à retourner lors de l'accès à la relation **imageable**. Dans ce cas, la colonne contiendrait soit **App\Models\Post** soit **App\Models\User**.

# *Relations polymorphes 1:1*

```
class Image extends Model
{
    /**
     * Get the parent imageable model (user or post).
     */
    public function imageable()
    {
        return $this->morphTo();
    }
}
```

# *Relations polymorphes 1:1*

```
class Post extends Model
{
    /**
     * Get the post's image.
     */
    public function image()
    {
        return $this->morphOne(Image::class, 'imageable');
    }
}
```

# *Relations polymorphes 1:1*

```
class User extends Model
{
    /**
     * Get the user's image.
     */
    public function image()
    {
        return $this->morphOne(Image::class, 'imageable');
    }
}
```

## *Relations polymorphes 1:1*

Une fois que votre table de base de données et vos modèles sont définis, vous pouvez accéder aux relations via vos modèles.

Par exemple, pour récupérer l'image du **post**, nous pouvons accéder à la propriété de relation dynamique **image** :

```
$post = Post::find(1);
```

```
$image = $post->image;
```

## *Relations polymorphes 1:1*

Vous pouvez récupérer le parent du modèle polymorphe en accédant au nom de la méthode qui effectue l'appel à **morphTo**. Dans ce cas, il s'agit de la méthode **imageable** du modèle **Image**. Nous allons donc accéder à cette méthode en tant que propriété de relation dynamique :

```
$image = Image::find(1);
```

```
$imageable = $image->imageable;
```

## *Relations polymorphes 1:1*

Si nécessaire, vous pouvez spécifier le nom des colonnes "id" et "type" utilisées par votre modèle enfant polymorphe. Si vous le faites, assurez-vous de toujours passer le nom de la relation comme premier argument de la méthode `morphTo`. En général, cette valeur doit correspondre au nom de la méthode, vous pouvez donc utiliser la constante `__FUNCTION__` de PHP.

```
public function imageable()  
{  
    return $this->morphTo(__FUNCTION__, 'imageable_type', 'imageable_id');  
}
```

# *Relations polymorphes 1:n*

Une relation polymorphe un-à-plusieurs est similaire à une relation un-à-plusieurs typique ; toutefois, le modèle enfant peut appartenir à plusieurs types de modèles à l'aide d'une seule association.



# *Relations polymorphes 1:n*

```
class Comment extends Model
{
    /**
     * Get the parent commentable model (post or video).
     */
    public function commentable()
    {
        return $this->morphTo();
    }
}
```

# *Relations polymorphes 1:n*

```
class Post extends Model
{
  /**
   * Get all of the post's comments.
   */
  public function comments()
  {
    return $this->morphMany(Comment::class, 'commentable');
  }
}
```

# *Relations polymorphes 1:n*

```
class Video extends Model
{
  /**
   * Get all of the video's comments.
   */
  public function comments()
  {
    return $this->morphMany(Comment::class, 'commentable');
  }
}
```

## *Relations polymorphes 1:n*

Une fois que votre table de base de données et vos modèles sont définis, vous pouvez accéder aux relations via les propriétés de relation dynamique de votre modèle.

```
$post = Post::find(1);  
  
foreach ($post->comments as $comment) {  
    //  
}
```

## *Relations polymorphes 1:n*

Vous pouvez également récupérer le parent d'un modèle enfant polymorphe en accédant au nom de la méthode qui effectue l'appel à `morphTo`.

```
$comment = Comment::find(1);  
  
$commentable = $comment->commentable;
```

La relation `commentable` sur le modèle `Commentaire` renverra une instance `Post` ou `Video`, en fonction du type de modèle parent du commentaire.

## *Relations un parmi plusieurs (polymorphe)*

Parfois, un modèle peut avoir de nombreux modèles liés, mais vous voulez facilement récupérer le "dernier" ou le "plus ancien" modèle lié de la relation.

Par exemple, un modèle User peut être lié à plusieurs modèles Image, mais vous souhaitez définir un moyen pratique d'interagir avec l'image la plus récente que l'utilisateur a téléchargée. Vous pouvez y parvenir en utilisant le type de relation **morphOne** combiné aux méthodes **ofMany**

```
public function latestImage()  
{  
    return $this->morphOne(Image::class, 'imageable')->latestOfMany();  
}
```

## *Relations un parmi plusieurs (polymorphe)*

De même, vous pouvez définir une méthode permettant de récupérer le modèle lié le plus "ancien", ou le premier, d'une relation :

```
/**  
 * Get the user's oldest image.  
 */  
public function oldestImage()  
{  
    return $this->morphOne(Image::class, 'imageable')->oldestOfMany();  
}
```

## *Relations un parmi plusieurs (polymorphe)*

De même, vous pouvez définir une méthode permettant de récupérer le modèle lié le plus "ancien", ou le premier, d'une relation :

```
public function oldestImage()  
{  
    return $this->morphOne(Image::class, 'imageable')->oldestOfMany();  
}
```

Par défaut, les méthodes **latestOfMany** et **oldestOfMany** récupèrent le modèle le plus récent ou le plus ancien en fonction de la clé primaire du modèle, qui doit être triable.

## *Relations un parmi plusieurs (polymorphe)*

Par défaut, les méthodes `latestOfMany` et `oldestOfMany` récupèrent le modèle le plus récent ou le plus ancien en fonction de la clé primaire du modèle, qui doit être triable. Cependant, il arrive que vous souhaitiez extraire un seul modèle d'une relation plus large en utilisant un critère de tri différent.

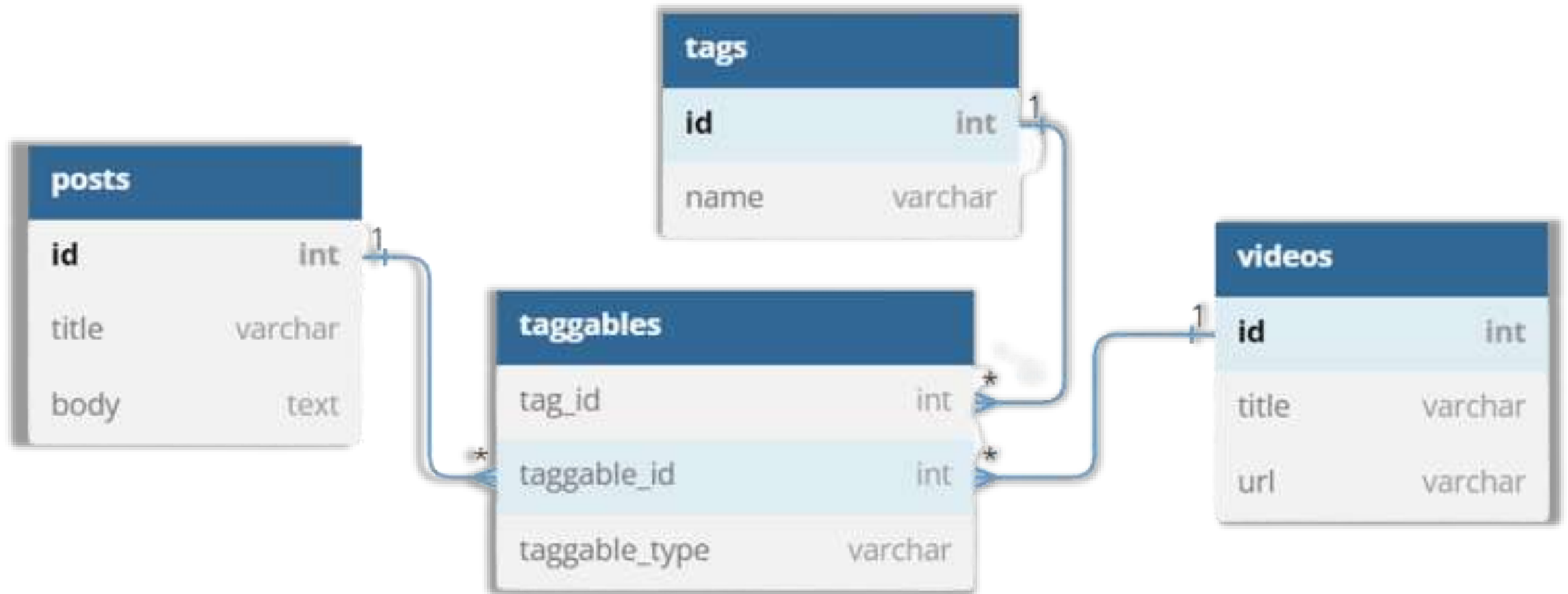
```
public function bestImage()  
{  
    return $this->morphOne(Image::class, 'imageable')->ofMany('likes', 'max');  
}
```

## *Relations plusieurs à plusieurs (polymorphe)*

Les relations polymorphes **Many-to-Many** sont légèrement plus compliquées que les relations "morph one" et "morph many".

Par exemple, un modèle **post** et un modèle **video** peuvent partager une relation polymorphe avec un modèle **tag**. L'utilisation d'une relation polymorphe de plusieurs à plusieurs dans cette situation permettrait à votre application de disposer d'une seule table **tags** uniques pouvant être associées à des **postes** ou à des **vidéos**.

# *Relations plusieurs à plusieurs (polymorphe)*



## *Relations plusieurs à plusieurs (polymorphe)*

Ensuite, nous sommes prêts à définir les relations sur les modèles. Les modèles **Post** et **Video** contiennent tous deux une méthode **tags** qui appelle la méthode **morphToMany** fournie par la classe de modèle Eloquent de base.

```
class Post extends Model
{
    public function tags()
    {
        return $this->morphToMany(Tag::class, 'taggable');
    }
}
```

## *Relations plusieurs à plusieurs (polymorphe)*

Ensuite, sur le modèle **Tag**, vous devez définir une méthode pour chacun de ses éventuels modèles parents.

```
class Tag extends Model
{
    public function posts()
    {
        return $this->morphedByMany(Post::class, 'taggable');
    }
    // Même chose pour videos()
}
```

## *Relations plusieurs à plusieurs (polymorphe)*

Une fois que votre table de base de données et vos modèles sont définis, vous pouvez accéder aux relations via vos modèles.

```
use App\Models\Post;

$post = Post::find(1);

foreach ($post->tags as $tag) {
    //
}
```

## *Relations plusieurs à plusieurs (polymorphe)*

Vous pouvez récupérer le parent d'une relation polymorphe à partir du modèle enfant polymorphe en accédant au nom de la méthode qui effectue l'appel à **morphedByMany**.

```
foreach ($tag->posts as $post) {  
    //  
}  
  
foreach ($tag->videos as $video) {  
    //  
}
```

## *Relations dynamiques*

Vous pouvez utiliser la méthode **resolveRelationUsing** pour définir les relations entre les modèles Eloquent au moment de l'exécution.

La méthode **resolveRelationUsing** accepte le nom de la relation souhaitée comme premier argument. Le second argument transmis à la méthode doit être une fermeture qui accepte l'instance de modèle et renvoie une définition de relation Eloquent valide.

```
Order::resolveRelationUsing('customer', function ($orderModel) {  
    return $orderModel->belongsTo(Customer::class, 'customer_id');  
});
```

## *Requête sur les relations*

Comme toutes les relations Eloquent sont définies par des méthodes, vous pouvez appeler ces méthodes pour obtenir une instance de la relation sans exécuter une requête pour charger les modèles liés.

```
$user = User::find(1);
```

```
$user->posts()->where('active', 1)->get();
```

# *Interroger l'existence d'une relation*

Lorsque vous récupérez des enregistrements de modèles, vous pouvez souhaiter limiter vos résultats en fonction de l'existence d'une relation. Pour ce faire, vous pouvez passer le nom de la relation aux méthodes `has` et `orHas`

```
use App\Models\Post;

// Retrieve all posts that have at least one comment...
$posts = Post::has('comments')->get();
```

## *Interroger l'absence d'une relation*

Lorsque vous récupérez des enregistrements de modèles, vous pouvez souhaiter limiter vos résultats en fonction de l'absence de relation. Pour ce faire, vous pouvez passer le nom de la relation aux méthodes `doesntHave` et `orDoesntHave`

```
$posts = Post::doesntHave('comments')->get();
```

# *Requête sur les relations Morph To*

Pour vérifier l'existence de relations "morph to", vous pouvez utiliser les méthodes `whereHasMorph` et `whereDoesntHaveMorph`.

```
$comments = Comment::whereHasMorph(  
    'commentable',  
    [Post::class, Video::class],  
    function (Builder $query) {  
        $query->where('title', 'like', 'code%');  
    }  
)->get();
```

# *Requête sur les relations Morph To*

Pour vérifier l'existence de relations "morph to", vous pouvez utiliser les méthodes `whereHasMorph` et `whereDoesntHaveMorph`.

```
$comments = Comment::whereDoesntHaveMorph(  
    'commentable',  
    Post::class,  
    function (Builder $query) {  
        $query->where('title', 'like', 'code%');  
    }  
)->get();
```

# *Comptage des modèles liés*

Vous souhaitez parfois compter le nombre de modèles liés pour une relation donnée sans avoir à charger les modèles. Pour ce faire, vous pouvez utiliser la méthode **withCount**. La méthode **withCount** place un attribut **{relation}\_count** sur les modèles résultants.

```
$posts = Post::withCount('comments')->get();  
  
foreach ($posts as $post) {  
    echo $post->comments_count;  
}
```

# *Comptage des modèles liés*

En passant un tableau à la méthode **withCount**, vous pouvez ajouter les "counts" de plusieurs relations ainsi que des contraintes supplémentaires aux requêtes.

```
$posts = Post::withCount(['votes', 'comments' => function (Builder $query) {  
    $query->where('content', 'like', 'code%');  
}])->get();
```

```
echo $posts[0]->votes_count;  
echo $posts[0]->comments_count;
```

# *Comptage des modèles liés*

En utilisant la méthode **loadCount**, vous pouvez charger un compte de relation après que le modèle parent ait été récupéré.

```
$book = Book::first();  
  
$book->loadCount('genres');
```

## *Comptage des modèles liés*

Si vous devez définir des contraintes de requête supplémentaires pour la requête de comptage, vous pouvez passer un tableau avec les clés des relations que vous souhaitez compter. Les valeurs du tableau doivent être des fermetures qui reçoivent l'instance du constructeur de requêtes.

```
$book->loadCount(['reviews' => function ($query) {  
    $query->where('rating', 5);  
}])
```

## *Autres fonctions agrégées*

En plus de la méthode `withCount`, Eloquent fournit les méthodes `withMin`, `withMax`, `withAvg`, `withSum` et `withExists`. Ces méthodes placent un attribut `{relation}_{fonction}_{column}` sur vos modèles résultants.

```
$posts = Post::withSum('comments', 'votes')->get();

foreach ($posts as $post) {
    echo $post->comments_sum_votes;
}
```

## *Autres fonctions agrégées*

Comme la méthode **loadCount**, des versions différées de ces méthodes sont également disponibles. Ces opérations d'agrégation supplémentaires peuvent être effectuées sur les modèles Eloquent qui ont déjà été récupérés.

```
$post = Post::first();
```

```
$post->loadSum('comments', 'votes');
```

# *Eager Loading*

Lors de l'accès aux relations Eloquent en tant que propriétés, les modèles liés sont chargés en "**Lazy Loading**". Cela signifie que les données de la relation ne sont pas réellement chargées avant le premier accès à la propriété. Cependant, Eloquent peut "**Eager Loading**" les relations au moment où vous interrogez le modèle parent.

```
$books = Book::with('author')->get();  
  
foreach ($books as $book) {  
    echo $book->author->name;  
}
```

## *Eager Loading*

Parfois, vous pouvez avoir besoin de charger plusieurs liens différents. Pour ce faire, il suffit de passer un tableau de relations à la méthode `with`.

```
$books = Book::with(['author', 'publisher'])->get();
```

Pour charger les relations d'une relation, vous pouvez utiliser la syntaxe "point".

```
$books = Book::with('author.contacts')->get();
```

# *Eager Loading*

Vous pouvez également spécifier des relations imbriquées chargées de en fournissant un tableau imbriqué à la méthode **with**, ce qui peut être pratique lors du chargement de plusieurs relations imbriquées.

```
$books = Book::with([  
    'author' => [  
        'contacts',  
        'publisher',  
    ],  
])->get();
```

## *Eager Loading*

Vous n'avez pas toujours besoin de toutes les colonnes des relations que vous récupérez. Pour cette raison, Eloquent vous permet de spécifier les colonnes de la relation que vous souhaitez récupérer.

```
$books = Book::with('author:id,name,book_id')->get();
```

## *Eager Loading*

Il peut arriver que vous souhaitiez toujours charger certaines relations lors de la récupération d'un modèle. Pour ce faire, vous pouvez définir une propriété `$with` sur le modèle.

```
protected $with = ['author'];
```

Si vous souhaitez supprimer un élément de la propriété `$with` pour une seule requête, vous pouvez utiliser la méthode `without` :

```
$books = Book::without('author')->get();
```

# *Lazy Eager Loading*

Parfois, vous pouvez avoir besoin de charger une relation après que le modèle parent ait déjà été récupéré.

Par exemple, cela peut être utile si vous devez décider dynamiquement de charger ou non des modèles liés :

```
$books = Book::all();  
  
if ($someCondition) {  
    $books->load('author', 'publisher');  
}
```

## *Insertion et mise à jour des modèles liés*

Eloquent fournit des méthodes pratiques pour ajouter de nouveaux modèles aux relations. Vous pouvez insérer un modèle lié en utilisant la méthode de **save**:

```
$comment = new Comment(['message' => 'A new comment.']);  
  
$post = Post::find(1);  
  
$post->comments()->save($comment);
```

Si vous devez sauvegarder plusieurs modèles liés, vous pouvez utiliser la méthode **saveMany**

## *Insertion et mise à jour des modèles liés*

Les méthodes `save` et `saveMany` persistent les instances de modèle données, mais n'ajoutent pas les modèles nouvellement persistés aux relations en mémoire déjà chargées sur le modèle parent. vous pouvez utiliser la méthode `refresh` pour recharger le modèle et ses relations.

```
$post->comments()->save($comment);
```

```
$post->refresh();
```

```
// All comments, including the newly saved comment...
```

```
$post->comments;
```

## *Insertion et mise à jour des modèles liés*

Si vous souhaitez sauvegarder votre modèle et toutes les relations qui lui sont associées, vous pouvez utiliser la méthode "**push**". Dans cet exemple, le modèle **Post** sera sauvegardé ainsi que ses commentaires et les auteurs des commentaires.

```
$post = Post::find(1);  
  
$post->comments[0]->message = 'Message';  
$post->comments[0]->author->name = 'Author Name';  
  
$post->push();
```

## *Insertion et mise à jour des modèles liés*

Vous pouvez utiliser la méthode **create**, qui accepte un tableau d'attributs pour créer un modèle et l'insère dans la base de données.

```
$post = Post::find(1);  
  
$comment = $post->comments()->create([  
    'message' => 'A new comment.',  
]);
```

## *Insertion et mise à jour des modèles liés*

Si vous souhaitez attribuer un modèle enfant à un nouveau modèle parent, vous pouvez utiliser la méthode **associate**.

```
$account = Account::find(10);  
  
$user->account()->associate($account);  
  
$user->save();
```

## *Insertion et mise à jour des modèles liés*

Pour supprimer un modèle parent d'un modèle enfant, vous pouvez utiliser la méthode **dissociate**. Cette méthode donne la valeur **null** à la clé étrangère de la relation.

```
$user->account()->dissociate();
```

```
$user->save();
```

## *Insertion et mise à jour des modèles liés*

Eloquent fournit également des méthodes pour rendre plus pratique le travail avec les relations **many-to-many**.

Par exemple, imaginons qu'un **utilisateur** puisse avoir plusieurs **rôles** et qu'un **rôle** puisse avoir plusieurs **utilisateurs**. Vous pouvez utiliser la méthode **attach** pour attacher un **rôle** à un **utilisateur** en insérant un enregistrement dans la table intermédiaire de la relation.

```
$user = User::find(1);
```

```
$user->roles()->attach($roleId);
```

## *Insertion et mise à jour des modèles liés*

Lorsque vous attachez une relation à un modèle, vous pouvez également transmettre un tableau de données supplémentaires à insérer dans la table intermédiaire.

```
$user->roles()->attach($roleId, ['expires' => $expires]);
```

## *Insertion et mise à jour des modèles liés*

Pour supprimer un enregistrement de relation n:n, utilisez la méthode **detach**. Cette méthode supprime l'enregistrement correspondant de la table intermédiaire, mais les deux modèles restent dans la base de données.

```
// Détacher un rôle unique de l'utilisateur...
```

```
$user->roles()->detach($roleId);
```

```
// Détacher tous les rôles de l'utilisateur...
```

```
$user->roles()->detach();
```

## *Insertion et mise à jour des modèles liés*

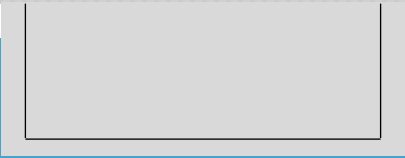
Vous pouvez également utiliser la méthode **sync** pour construire des associations de plusieurs à plusieurs. La méthode **sync** accepte un tableau d'identifiants à placer dans la table intermédiaire. Tous les ID qui ne sont pas dans le tableau donné seront supprimés du tableau intermédiaire. Ainsi, une fois cette opération terminée, seuls les identifiants du tableau donné existeront dans la table intermédiaire :

```
$user->roles()->sync([1, 2, 3]);
```

## *Insertion et mise à jour des modèles liés*

Si vous devez mettre à jour une ligne existante dans le tableau intermédiaire de votre relation, vous pouvez utiliser la méthode **updateExistingPivot**. Cette méthode accepte la clé étrangère de l'enregistrement intermédiaire et un tableau d'attributs à mettre à jour

```
$user = User::find(1);  
  
$user->roles()->updateExistingPivot($roleId, [  
    'active' => false,  
]);
```



# *SÉRIALISATION*

# *Introduction*

Lorsque vous construisez des API à l'aide de Laravel, vous aurez souvent besoin de convertir vos modèles et vos relations en tableaux ou en JSON. Eloquent inclut des méthodes pratiques pour effectuer ces conversions, ainsi que pour contrôler les attributs qui sont inclus dans la représentation sérialisée de vos modèles.

## *Sérialisation vers des tableaux*

Pour convertir un modèle et ses relations chargées en un tableau, vous devez utiliser la méthode **toArray**. Cette méthode est réursive, de sorte que tous les attributs et toutes les relations (y compris les relations de relations) seront convertis en tableaux :

```
use App\Models\User;  
  
$user = User::with('roles')->first();  
  
return $user->toArray();
```

## *Sérialisation vers des tableaux*

La méthode `attributesToArray` peut être utilisée pour convertir les attributs d'un modèle en un tableau, mais pas ses relations :

```
$user = User::first();  
  
return $user->attributesToArray();
```

## *Sérialisation vers des tableaux*

Vous pouvez également convertir des collections entières de modèles en tableaux en appelant la méthode **toArray** sur l'instance de la collection :

```
$users = User::all();  
  
return $users->toArray();
```

# *Sérialisation en JSON*

Pour convertir un modèle en JSON, vous devez utiliser la méthode `toJson`. Comme `toArray`, la méthode `toJson` est récursive, donc tous les attributs et les relations seront convertis en JSON.

```
$user = User::find(1);  
  
return $user->toJson();  
  
return $user->toJson(JSON_PRETTY_PRINT);
```

# *Sérialisation en JSON*

Vous pouvez également convertir un modèle ou une collection en une chaîne de caractères, ce qui appelle automatiquement la méthode `toJson` sur le modèle ou la collection :

```
return (string) User::find(1);
```

# *Sérialisation en JSON*

Puisque les modèles et les collections sont convertis en JSON lorsqu'ils sont convertis en chaîne, vous pouvez renvoyer des objets Eloquent directement depuis les routes ou les contrôleurs de votre application. Laravel sérialise automatiquement vos modèles et collections Eloquent en JSON lorsqu'ils sont retournés par les routes ou les contrôleurs :

```
Route::get('users', function () {  
    return User::all();  
});
```

# *Masquer les attributs de JSON*

Parfois, vous souhaitez limiter les attributs, tels que les mots de passe, qui sont inclus dans le tableau ou la représentation JSON de votre modèle. Pour ce faire, ajoutez une propriété **\$hidden** à votre modèle. Les attributs figurant dans le tableau de la propriété **\$hidden** ne seront pas inclus dans la représentation sérialisée de votre modèle :

```
class User extends Model
{

    protected $hidden = ['password'];

}
```

# *Masquer les attributs de JSON*

Alternativement, vous pouvez utiliser la propriété `visible` pour définir une "liste autorisée" d'attributs qui doivent être inclus dans le tableau de votre modèle et la représentation JSON. Tous les attributs qui ne sont pas présents dans le tableau `$visible` seront masqués lorsque le modèle sera converti en tableau ou en JSON :

```
class User extends Model
{
    protected $visible = ['first_name', 'last_name'];
}
```